

MODULE 16

Improving Performance of Deep Learning Models

Topics Covered

- Lectures 1–2 — Performance Framework, Early Stopping & Data Scaling
- Lectures 3–4 — Dropout & Regularisation
- Lectures 5–6 — Batch Normalisation
- Lectures 7–8 — Activation Functions
- Lectures 9–11 — Optimisers

Lectures 1–2: Performance Framework, Early Stopping & Data Scaling

Diagnosing Model Performance

Before applying any fix, diagnose the problem. The training vs validation loss curve tells you everything:

Scenario	Diagnosis Fix
High train loss + high val loss	Underfitting (high bias) — model too simple. Fix: more layers/neurons, train longer, better features.
Low train loss + high val loss	Overfitting (high variance) — model memorised training data. Fix: dropout, regularisation, more data, early stopping.
Both losses high & not decreasing	Learning rate too low, vanishing gradients, or wrong architecture.
Loss oscillates wildly	Learning rate too high or batch size too small. Fix: reduce LR, increase batch.

Performance Improvement Checklist

Category	Techniques
More / better data	Data augmentation, gather more samples, fix label errors, SMOTE for imbalance
Regularisation	L1 (Lasso), L2 (Ridge/Weight decay), Dropout, Early Stopping
Better architecture	More/fewer layers, skip connections, attention mechanisms
Normalisation	Batch Norm, Layer Norm, Group Norm — stabilise training
Better optimiser	Switch from SGD → Adam; tune learning rate; use LR scheduler
Data scaling	StandardScaler, MinMaxScaler — critical for gradient-based methods

Early Stopping

Early stopping monitors validation loss during training and halts when it stops improving — preventing the model from memorising training data.

```
from tensorflow.keras.callbacks import EarlyStopping

es = EarlyStopping(
    monitor='val_loss',          # watch validation loss
    patience=10,                 # stop after 10 epochs of no improvement
    min_delta=0.001,             # minimum change to count as improvement
    restore_best_weights=True    # revert to best epoch weights
)

history = model.fit(
    X_train, y_train,            # set high — early stopping will halt
    epochs=500,
    validation_split=0.2,
    callbacks=[es]
)

print(f'Stopped at epoch: {es.stopped_epoch}')
```

Data Scaling

Neural networks use gradient descent — features on very different scales cause uneven gradient updates and slow convergence. Always scale inputs before training.

Scaler	Formula When to Use
StandardScaler	$z = (x - \mu) / \sigma \rightarrow \text{mean}=0, \text{std}=1$. Best for normally distributed data. Most common.
MinMaxScaler	$x' = (x - \min) / (\max - \min) \rightarrow \text{range } [0, 1]$. Use when bounded range needed (e.g. image pixels).
RobustScaler	Uses median & IQR instead of mean & std. Best when data has outliers.
LogTransform	$x' = \log(x+1)$. For heavily skewed features (income, word counts).

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit on train only
X_test_scaled = scaler.transform(X_test)       # transform only

# For image data - normalise pixel values to [0,1]
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
```

□ Data leakage

Always fit the scaler on training data ONLY.

Fitting on the full dataset (including test) leaks test statistics into training — inflating performance metrics.

Lectures 3–4: Dropout & Regularisation

Dropout

Dropout randomly deactivates a fraction of neurons during each training step. This prevents neurons from co-adapting and forces the network to learn redundant, more robust representations.

□ Analogy

Imagine studying for an exam where random team members are absent each session.

Each person must know the full material independently — no one can rely on another.

The team (network) becomes more robust as a result.

```
from tensorflow.keras.layers import Dense, Dropout

model = tf.keras.Sequential([
    Dense(256, activation='relu'),
    Dropout(0.4),           # drop 40% of neurons randomly during training
    Dense(128, activation='relu'),
    Dropout(0.3),           # drop 30%
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(1, activation='sigmoid')
])

# Important: Dropout is ONLY active during training
# During inference (predict/evaluate), all neurons are active
```

```
# Keras handles this automatically via the 'training=True/False' flag
```

Dropout Rate	Effect Recommended For
0.1 – 0.2	Light regularisation — small datasets or shallow networks
0.3 – 0.5	Standard — most hidden layers in medium-large networks
0.5	Classic recommendation (Hinton et al.) for fully connected layers
> 0.6	Too aggressive — model underfits; rarely used

L1 & L2 Regularisation

Regularisation adds a penalty term to the loss function that discourages large weights, preventing the model from over-relying on any single feature.

Method	Penalty Effect Use Case
L2 (Ridge / Weight Decay)	$\lambda \sum w_i^2$ — penalises large weights. Spreads weight across features. Most common in DL.
L1 (Lasso)	$\lambda \sum w_i $ — drives some weights to exactly 0 (sparse). Built-in feature selection.
L1 + L2 (Elastic Net)	$\lambda_1 \sum w_i + \lambda_2 \sum w_i^2$ — combines both. Best when many features, some irrelevant.

```
from tensorflow.keras import regularizers

# L2 regularisation
Dense(128, activation='relu',
      kernel_regularizer=regularizers.l2(0.01))

# L1 regularisation
Dense(128, activation='relu',
      kernel_regularizer=regularizers.l1(0.001))

# L1 + L2 combined
Dense(128, activation='relu',
      kernel_regularizer=regularizers.l1_l2(l1=0.001, l2=0.01))

# In PyTorch: use weight_decay in optimiser (equivalent to L2)
optimizer = torch.optim.AdamW(model.parameters(),
                                lr=3e-4, weight_decay=0.01)
```

Regularisation Technique	Prevents Best When
L2 (Weight Decay)	Large individual weights Always a good default
L1	Non-zero weights on irrelevant features Many features, need selection
Dropout	Co-adaptation of neurons Deep fully-connected layers
Early Stopping	Training past generalisation point Always use as first line of defence
Data Augmentation	Overfitting due to small dataset Image, audio, text tasks

Lectures 5–6: Batch Normalisation

The Problem Batch Norm Solves

As training progresses, the distribution of inputs to each layer shifts because earlier layer weights keep changing. This is called Internal Covariate Shift. Each layer must constantly adapt to a moving target, slowing training dramatically.

How Batch Normalisation Works

For each mini-batch, BatchNorm normalises the layer's inputs to have approximately zero mean and unit variance, then applies learnable scale (γ) and shift (β) parameters:

```
# For a mini-batch B = {x1, x2, ..., xm}:
```

$$\mu_B = (1/m) \sum x_i$$

```
# Step 1: Compute batch mean
```

$$\sigma^2_B = (1/m) \sum (x_i - \mu_B)^2$$

```
# Step 2: Compute batch variance
```

$$\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)}$$

← $\epsilon=1e-5$ prevents divide-by-zero

```
# Step 3: Normalise
```

```
# Step 4: Scale and shift (learnable  $\gamma$  and  $\beta$ )
```

$$y_i = \gamma \cdot \hat{x}_i + \beta$$

```
#  $\gamma$  and  $\beta$  are learned during training - allow the network to
```

```
# undo normalisation if that's optimal for the task
```

BatchNorm in Keras

```
from tensorflow.keras.layers import Dense, BatchNormalization, Activation
```

```
# Recommended placement: Dense → BatchNorm → Activation
```

```
model = tf.keras.Sequential([
```

```
    Dense(256, kernel_initializer='he_normal'),
```

```
    BatchNormalization(),
```

```
    Activation('relu'),
```

```
    Dense(128, kernel_initializer='he_normal'),
```

```
    BatchNormalization(),
```

```
    Activation('relu'),
```

```
    Dense(10, activation='softmax')
```

```
])
```

```
# Training vs inference behaviour:
```

```
# Training: uses current batch mean & variance
```

```
# Inference: uses running mean & variance (accumulated during training)
```

```
# Keras handles this automatically
```

Benefit	Details
Faster training	Allows higher learning rates — loss landscape becomes smoother
Reduces sensitivity to init	Less critical to choose perfect weight initialisation
Mild regularisation	Noise from batch statistics acts as a regulariser (less need for Dropout)
Mitigates vanishing gradients	Keeps activations in non-saturating region of activation functions

Batch Norm in Generative AI Models

Large language models (GPT, LLaMA, Mistral) use Layer Normalisation instead of Batch Norm. The reason: LLMs process variable-length sequences and sometimes batch size = 1 — batch statistics become unreliable.

Normalisation	Normalises Over Used In
Batch Norm	Batch dimension (across samples in a batch) → CNNs, feedforward networks
Layer Norm	Feature dimension (across all features of ONE sample) → Transformers, GPT, BERT, LLaMA
Group Norm	Groups of channels → Object detection, small batch tasks
RMS Norm	Root mean square normalisation (no mean subtraction) → LLaMA, Mistral (faster than LayerNorm)

```

# Layer Normalisation (used in Transformers)
from tensorflow.keras.layers import LayerNormalization

# In a Transformer block:
x = MultiHeadAttention(...)(x, x)
x = LayerNormalization(epsilon=1e-6)(x)    # after attention
x = FeedForward()(x)
x = LayerNormalization(epsilon=1e-6)(x)    # after FFN

# RMSNorm (LLaMA) - simpler, faster
class RMSNorm(tf.keras.layers.Layer):
    def call(self, x):
        rms = tf.sqrt(tf.reduce_mean(tf.square(x), axis=-1, keepdims=True))
        return x / (rms + 1e-6)

```

Lectures 7–8: Activation Functions

Why Activation Functions Are Essential

Without activation functions, stacking multiple layers is equivalent to a single linear layer — no matter how many layers you add. Activations introduce non-linearity, allowing the network to learn complex, curved decision boundaries.

```

# Without activation: output = W3(W2(W1·X)) = (W3·W2·W1)·X = W_combined·X
# No matter how many layers - still just one linear transformation!

# With ReLU: each layer can represent a different non-linear piece
# → network can approximate ANY continuous function (Universal Approximation Theorem)

```

Sigmoid

Property	Value
Formula	$\sigma(z) = 1 / (1 + e^{-z})$
Range	(0, 1)
Derivative	$\sigma(z) \cdot (1 - \sigma(z)) \leq 0.25$ — causes vanishing gradients
Use case	Output layer for binary classification ONLY. Avoid in hidden layers.

Property	Value
Problem	Saturates at extremes → near-zero gradients → slow learning

Tanh

Property	Value
Formula	$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$
Range	$(-1, 1)$ — zero-centred (advantage over sigmoid)
Derivative	$1 - \tanh^2(z) \leq 1.0$ — still saturates
Use case	Hidden layers in RNNs/LSTMs. Better than sigmoid but still saturates.

ReLU (Rectified Linear Unit)

Property	Value
Formula	$\text{ReLU}(z) = \max(0, z)$
Range	$[0, \infty)$
Derivative	1 if $z > 0$, else 0 — no saturation for positive inputs
Use case	Default for hidden layers in most deep networks
Advantage	Computationally cheap; no vanishing gradient for positive values; sparse activations
Problem	Dying ReLU: if $z < 0$ for all inputs, neuron outputs 0 forever — weight never updates

ReLU Variants

Variant	Formula Fixes Notes
Leaky ReLU	$\max(0.01z, z)$ — small gradient for negatives. Fixes dying ReLU.
PReLU	$\max(\alpha z, z)$ — α is learnable. Adaptive slope for negatives.
ELU	z if $z > 0$ else $\alpha(e^z - 1)$ — smooth negative region, mean closer to 0.
SELU	Self-normalising ELU — keeps activations normalised without BatchNorm. Used in deep FCNs.
GELU	$z \cdot \Phi(z)$ where Φ = Gaussian CDF. Smooth probabilistic gating. Used in BERT, GPT, ViT.
SiLU/Swish	$z \cdot \sigma(z)$ — smooth, non-monotonic. Used in LLaMA, EfficientNet.
Mish	$z \cdot \tanh(\text{softplus}(z))$ — smooth, slight negative values. Used in YOLO.
<pre> from tensorflow.keras.layers import Activation, Dense import tensorflow as tf # Standard activations Dense(128, activation='relu') Dense(128, activation='leaky_relu') Dense(128, activation='elu') Dense(128, activation='selu') # GELU — used in GPT/BERT Dense(128, activation='gelu') </pre>	

Variant	Formula Fixes Notes
<pre># SiLU/Swish — used in LLaMA Dense(128, activation='swish') # Custom Mish def mish(x): return x * tf.math.tanh(tf.math.softplus(x)) Dense(128, activation=mish)</pre>	

Activation Function Summary

Activation	Where It's Used Today
ReLU	Default hidden layers — CNNs, basic MLPs
Leaky ReLU	When dying ReLU is a problem — object detection (YOLO)
GELU	Transformers — BERT, GPT-2, GPT-3, ViT
SiLU/Swish	LLMs — LLaMA, Mistral, EfficientNet
Sigmoid	Binary classification output layer only
Softmax	Multi-class classification output layer
Tanh	RNN/LSTM hidden states; GAN generators

Lectures 9–11: Optimisers

What is an Optimiser?

An optimiser is the algorithm that updates the network's weights to minimise the loss function. It takes the gradients computed by backprop and decides how much to change each weight.

Vanilla SGD

```
# W = W - α × ∂L/∂W

# Problems:
# - Same LR for all parameters
# - Noisy updates (high variance)
# - Slow convergence on flat or ravine-shaped loss surfaces
# - Gets stuck in saddle points

optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
```

SGD + Momentum

Adds a velocity term — the optimiser accumulates movement in the gradient direction, dampening oscillations and speeding up convergence.

```
# v = β · v1 + α · ∂L/∂W    (velocity accumulation)
# W = W - v

# β = 0.9 is standard (remember 90% of last velocity)
```



```
# Like a ball rolling downhill — builds momentum

optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)

# Nesterov momentum — looks ahead before updating (often better)
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01,
                                     momentum=0.9, nesterov=True)
```

AdaGrad (Adaptive Gradient)

Adapts the learning rate for each parameter individually — parameters that receive large gradients get smaller learning rates; rare parameters get larger learning rates. Great for sparse data and NLP.

```
#  $G_t = G_{t-1} + (\partial L / \partial W)^2$  (accumulate squared gradients)
#  $W = W - (\alpha / \sqrt{G_t + \epsilon}) \times \partial L / \partial W$ 

# Effect:
# - Frequent features (large  $G_t$ ) → small effective LR
# - Rare features (small  $G_t$ ) → large effective LR

# Problem:  $G_t$  grows monotonically → LR shrinks to near 0 → training stops

optimizer = tf.keras.optimizers.Adagrad(learning_rate=0.01)
```

RMSprop (Root Mean Square Propagation)

Fixes AdaGrad's shrinking learning rate by using an exponential moving average of squared gradients instead of accumulating them:

```
#  $s_t = \beta \cdot s_{t-1} + (1-\beta) \cdot (\partial L / \partial W)^2$  (decaying average — NOT accumulation)
#  $W = W - (\alpha / \sqrt{s_t + \epsilon}) \times \partial L / \partial W$ 

#  $\beta = 0.9$  typical — focuses on recent gradients, forgets old ones
# LR stays stable throughout training (fixes AdaGrad's problem)

optimizer = tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)

# Good for: RNNs, LSTMs — originally designed for non-stationary objectives
```

Adam (Adaptive Moment Estimation)

Adam combines SGD+Momentum (first moment) and RMSprop (second moment). The most widely used optimiser in deep learning.

```
# First moment (momentum):  $m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot \partial L / \partial W$ 
# Second moment (RMSprop):  $v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot (\partial L / \partial W)^2$ 

# Bias correction (fixes initialisation at 0):
#  $\hat{m}_t = m_t / (1 - \beta_1^t)$ 
#  $\hat{v}_t = v_t / (1 - \beta_2^t)$ 

# Update rule:
#  $W = W - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ 

# Default hyperparameters (rarely need tuning):
#  $\beta_1 = 0.9$   $\beta_2 = 0.999$   $\epsilon = 1e-7$   $\alpha = 0.001$ 
```

```
optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.001,
    beta_1=0.9,
    beta_2=0.999,
    epsilon=1e-7
)
```

AdamW — Adam for GenAI Models

AdamW decouples weight decay from the gradient update — applying L2 regularisation directly to weights rather than through the gradient. Used to train GPT, BERT, LLaMA, Mistral, and most modern LLMs.

```
# Adam:  W = W - α · m̂ / (√v̂ + ε) - α · λ · W_gradient  ← L2 via gradient (incorrect)
# AdamW: W = W - α · m̂ / (√v̂ + ε) - α · λ · W          ← L2 directly on weight (correct)

optimizer = tf.keras.optimizers.AdamW(
    learning_rate=3e-4,
    weight_decay=0.01  # standard for LLM pre-training
)

# PyTorch (more commonly used for LLMs)
import torch
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=3e-4,
    betas=(0.9, 0.999),
    weight_decay=0.01
)
```

Optimiser Comparison

Optimiser	Key Idea Best For Used In
SGD	Fixed LR for all params Simple convex problems Baseline
SGD+Momentum	Velocity accumulation CV tasks with LR scheduling ResNet, VGG training
AdaGrad	Adaptive LR, cumulative Sparse NLP features Older NLP models
RMSprop	Adaptive LR, decaying RNNs, non-stationary LSTMs, RL
Adam	Momentum + RMSprop Most DL tasks CNNs, MLPs, GANs
AdamW	Adam + proper weight decay LLMs, Transformers GPT, BERT, LLaMA, Mistral

Learning Rate Scheduling

A fixed learning rate is rarely optimal. Schedulers reduce the LR over training for better convergence:

```
import tensorflow as tf

# Step decay - reduce by factor every N epochs
lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=1e-3,
    decay_steps=1000,
    decay_rate=0.9)

# Cosine decay - used in LLM training
```

```
lr_schedule = tf.keras.optimizers.schedules.CosineDecay(  
    initial_learning_rate=3e-4,  
    decay_steps=100000)  
  
# Warmup + cosine (standard for transformers)  
# Linear warmup for first 4% of steps, then cosine decay  
  
model.compile(optimizer=tf.keras.optimizers.AdamW(  
    learning_rate=lr_schedule, weight_decay=0.01))
```

End of Module 16 Notes — Generative AI Course